

Cyberdeck Build

The full written lesson — power budgeting, display selection, assembly & thermals, real-world testing.

1. Power budgeting: measure, don't guess

A cyberdeck's runtime is decided before you ever touch a soldering iron — it's arithmetic, and the arithmetic is only as good as the numbers you feed it.

- Datasheet/nameplate current is a **ceiling**, not what your specific build draws. A Pi 5's quoted peak current happens under sustained full CPU load with every peripheral active — your actual average draw while reading a terminal is lower, but a video call is higher.
- Measure **real draw** with an inline USB power meter, under the load you'll actually run, not idle and not synthetic peak.
- A "10,000mAh" USB power bank does **not** deliver 10,000mAh at 5V to your load. The cells inside store energy at ~3.7V; a boost converter steps that up to 5V, and that conversion loses roughly 15–20% to heat. Budget against *usable* capacity, not the number printed on the case.

The trap: summing every component's absolute maximum rated current and concluding the build is impossible, or trusting idle-state figures and being shocked when real runtime is a third of the estimate. Neither extreme is your actual number — measure the real workload.

Worked: Pi 5 under typical desktop-ish load (~900mA) + a 7" touchscreen with backlight on (~300mA) + a wireless keyboard/mouse dongle (~50mA) ≈ **1250mA** total. A 10,000mAh bank at 85% conversion efficiency gives ≈8500mAh usable. Runtime ≈ 8500 / 1250 ≈ **6.8 hours** — and that's still optimistic versus a workload with active wifi transfers or a brighter screen.

2. Choosing and configuring a display

The display is the single component most likely to eat a weekend if you pick wrong, and the difference between DSI and HDMI touchscreens is bigger than it looks in a spec sheet.

- **DSI (Display Serial Interface)** connects through the Pi's dedicated display connector via a flat ribbon cable — one connection, official displays, the best driver support of any option.
- **HDMI touchscreens** need *two* connections: HDMI for video, a separate USB cable for touch input. Generic panels often need manual `config.txt` overlay entries to get resolution/timing right.
- `dtoverlay=` lines in `/boot/firmware/config.txt` configure non-standard display timing and rotation.

The trap: assuming rotating the display also rotates the touch input. `display_lcd_rotate=` flips the video, but touch coordinates are a *separate* mapping — without its own transform matrix, a rotated screen reports touches as if it were still in the original orientation. Video working $\neq$ touch working.

Worked: the Build Order's own rule — "get the display working before committing to the enclosure" — exists because a DSI ribbon cable and an HDMI+USB cable pair take up genuinely different physical space and routing. Deciding the enclosure's internal layout before the display is confirmed working means guessing at a dimension you don't actually have yet.

3. Assembly order and thermal management

A cyberdeck fails for mechanical reasons as often as electrical ones, and both failure modes follow the same pattern: skipping a step because it seems like it'll obviously be fine.

- **Electronics before enclosure, always.** Test the full stack loose on a bench before any cutting, printing, or drilling. A wiring or driver problem found after the enclosure is built means redoing enclosure work, not just re-wiring.
- **Strain relief:** a wire routed taut looks fine on day one. Repeated flexing stresses a taut connector joint until it fails, often intermittently before it fails completely. Route every wire with a small service loop.
- **Thermal throttling:** the Pi automatically downclocks itself when it gets too hot — a sealed enclosure with no airflow will trigger this, and a throttled Pi is a slower, less responsive machine, not a broken one, which makes the cause easy to misdiagnose as a software problem.

The trap: gluing in a heatsink and assuming the thermal problem is solved. A heatsink only works by moving heat into air that's actually flowing past its fins — inside a fully sealed box, it just becomes a slightly larger hot mass. You need a vent, a fan, or both.

Worked: a realistic build order — bench-test everything loose, confirm display + touch + battery runtime, *then* design the enclosure around the confirmed-working layout with vent placement decided by where heat actually needs to go.

4. Testing on battery and trimming the OS

The build isn't done when it boots on the bench — it's done when you've verified it does what you designed it to do, under real conditions, on battery.

- **Measured endurance is usually shorter than calculated endurance.** Lesson 1's math is a planning estimate, not a guarantee — wifi scanning, screen brightness, and background processes all add draw the calculation didn't necessarily account for precisely.
- **Trim the OS for the hardware you actually have.** Raspberry Pi OS Lite plus a minimal window manager, or a single kiosk-mode application, both reduce power and thermal load meaningfully.
- **Disable what you don't need.** The systemd skills from Raspberry-Pi-Tinkering apply directly here — a background service you're not using is drawing power and CPU for nothing.

The trap: treating the Lesson 1 power budget as the final answer instead of a planning estimate, and being surprised when real-world runtime comes in lower. The calculation catches a wildly undersized battery before buying it — it doesn't replace actually timing a real discharge.

Worked: calculated 6.8 hours from Lesson 1's numbers; measured real runtime under normal use came in at 5.2 hours because of intermittent wifi activity the static calculation didn't model. Budget a safety margin, and treat the calculated figure as a floor, not a promise.